
UNIVERSITY OF DHAKA

# RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing

Submitted by

Class Roll: 50028

Registration No: H-55

Class Roll: 50040

Registration No: H-49

Submitted to the

Department of Computer Science and Engineering

University of Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of

Professional Masters in Information and Cyber Security (PMICS).

# Declaration Form


University of Dhaka

The work in this report titled "**RedGrid: Dependency-Constrained UCB Exploration for Autonomous Penetration Testing**" represents the original work of the undersigned, carried out under the supervision of Dr. Md. Abdur Razzaque. The authors further declare that where information draws on the literature, the original sources have received adequate citation and referencing. The authors did not submit any part of this report earlier for another degree or diploma at this or any other institution.

Name of the Candidate
Nishan Paul

Name of the Candidate
Md Rakibur Rahman

Dr. Md. Abdur Razzaque
Professor and Chair
Department of Computer Science and Engineering
University of Dhaka

*"To conquer fear, you must become fear."*

Ra's al Ghul, Batman Begins

UNIVERSITY OF DHAKA

# *Abstract*

Autonomous penetration testing agents powered by large language models have shown rapid progress in recent years, yet published evaluations consistently point to two recurring weaknesses: agents explore candidate vulnerabilities too narrowly, and they struggle to reason about the prerequisite relationships between different stages of an attack. These weaknesses limit how far current systems can operate reliably against realistic, multi-step web, API, and network targets without close human supervision.

This report documents the first stage of an investigation into whether explicitly modeling the dependency relationships between attack steps — rather than treating candidate vulnerabilities as an unordered list — can improve how autonomous agents explore and rank their actions during a penetration test. As an initial step, the authors reviewed a body of recent literature on LLM-based offensive security agents, covering single-agent systems, multi-agent orchestration frameworks, planning-augmented approaches, and the benchmarks used to assess them. Two recurrent failure modes — insufficient exploration breadth and weak prerequisite reasoning — surfaced across independently developed systems. These observations motivate the research direction described in this report, which goes by the name RedGrid.

At this stage, the work is preliminary: a structured review of the current state of autonomous penetration testing research and an early architectural direction motivated by the gaps observed in the literature. The remaining work over the coming months focuses on refining this direction, implementing and assessing it against established benchmarks, and determining whether dependency-aware exploration meaningfully improves autonomous exploitation performance.

# *Acknowledgements*

Sincere gratitude goes to the supervisor, Dr. Md. Abdur Razzaque, Professor and Chair, Department of Computer Science and Engineering, University of Dhaka, for agreeing to guide this work and for the time he has already invested in it at such an early stage. His feedback pushed the authors to move beyond simply summarizing existing autonomous penetration testing systems and to instead ask what specific gap in that body of work this research could realistically address. That question shaped the direction this thesis has taken so far, and will continue to shape the work as it progresses over the coming months.

Acknowledgement also goes to the wider research community whose published work — spanning benchmark design, agent architectures, and empirical evaluations in offensive security — provided the foundation on which this thesis rests. Progress over the next six months depends heavily on the groundwork laid by that literature.

# Contents

# List of Figures

# List of Tables

# List of Algorithms

| **Acronym** | **W**hat (it) **S**tands **F**or |
|---|---|
| **ADM** | **A**ttack **D**ecision-**M**aking |
| **AI** | **A**rtificial **I**ntelligence |
| **API** | **A**pplication **P**rogramming **I**nterface |
| **CVE** | **C**ommon **V**ulnerabilities and **E**xposures |
| **CVSS** | **C**ommon **V**ulnerability **S**coring **S**ystem |
| **CTF** | **C**apture **T**he **F**lag |
| **DAG** | **D**irected **A**cyclic **G**raph |
| **EL** | **E**nvironmental **L**ayer |
| **GPT** | **G**enerative **P**re-trained **T**ransformer |
| **LLM** | **L**arge **L**anguage **M**odel |
| **PDDL** | **P**lanning **D**omain **D**efinition **L**anguage |
| **RCE** | **R**emote **C**ode **E**xecution |
| **ReAct** | **Re**ason + **Act** (agent framework) |
| **REST** | **Re**presentational **S**tate **T**ransfer |
| **SQL** | **S**tructured **Q**uery **L**anguage |
| **SQLi** | **SQL** **I**njection |
| **SSRF** | **S**erver-**S**ide **R**equest **F**orgery |
| **SSTI** | **S**erver-**S**ide **T**emplate **I**njection |
| **UCB** | **U**pper **C**onfidence **B**ound |
| **VAPT** | **V**ulnerability **A**ssessment and **P**enetration **T**esting |
| **VDG** | **V**ulnerability **D**ependency **G**raph |
| **WAF** | **W**eb **A**pplication **F**irewall |
| **XSS** | **C**ross-**S**ite **S**cripting |

*Dedicated to those who asked the harder question.*

# Chapter 1

# Introduction

## 1.1 Background and Motivation

### 1.1.1 Cybersecurity and Vulnerability Assessment

Modern organisations depend on networked software systems for nearly every operational function, and the security of those systems has become a central concern. A security vulnerability is a flaw in software, configuration, or design that an attacker can exploit to gain unauthorised access, execute arbitrary code, disrupt service, or exfiltrate data. Vulnerabilities are catalogued in the Common Vulnerabilities and Exposures (CVE) database after disclosure; the interval between disclosure and remediation deployment frequently extends to months in production environments, leaving systems exposed to any attacker who can reconstruct an exploit from the published description.

**Vulnerability Assessment and Penetration Testing (VAPT)** is the practice of proactively identifying and demonstrating these weaknesses before an adversary does. A penetration test is an authorised, systematic attempt to discover and exploit weaknesses in a target system, typically following a structured workflow: initial reconnaissance to understand the target's attack surface, identification of candidate weaknesses, exploitation of confirmed vulnerabilities, and reporting of findings with

remediation guidance. VAPT is widely recognised as an essential security practice, and demand for it has grown considerably as organisations increase their exposure through cloud services, APIs, and web applications.

### 1.1.2 Why Conventional VAPT Has Limits

Despite its importance, conventional penetration testing faces practical constraints. Skilled human testers are scarce and expensive: a thorough engagement against a moderately complex target demands days of expert effort and costs can reach thousands of dollars per assessment [4]. Scheduled, periodic testing leaves gaps during which newly introduced vulnerabilities remain undetected. Automated scanning tools — such as network port scanners and web application vulnerability scanners — can partially fill this gap, but they operate on fixed rule sets and signature databases and lack the contextual reasoning needed to chain together exploits. The practical limits of these tools are not merely theoretical: Fang et al. [5] tested the two most widely deployed open-source scanners (ZAP and Metasploit) against a benchmark of 15 real-world one-day vulnerabilities and found that both achieved 0% success, whereas a capable LLM agent succeeded on the same targets. Scanners cannot adapt to novel application logic, reason about prerequisite relationships between attack steps, or operate across the breadth of a modern attack surface without human guidance.

These limitations have driven a recent and rapid research interest in applying LLM-based agents to automate parts of the VAPT process — an interest that has produced a substantive body of work since 2023.

### 1.1.3 Large Language Models and AI Agents

Large Language Models (LLMs) are deep learning models trained on large corpora of text that have demonstrated a remarkable ability to follow instructions, reason through multi-step problems, write and execute code, and interact with external tools.

An **LLM agent** extends a base language model with the ability to take actions in an environment: calling tools (such as a web browser, a terminal, or an API), observing the results, and iterating toward a goal. The simplest agents follow a *Reason + Act* (ReAct) loop — reason about the current state, decide on an action, observe the outcome, and repeat. More sophisticated designs introduce hierarchical planning, role-specialised sub-agents, persistent memory, and structured validation.

A finding that recurs independently across six systems in the VAPT literature — and that directly shapes the design choices in this thesis — is that **architecture, not model scale, is the dominant variable in autonomous exploitation performance** [6]. Structured pipelines running comparatively inexpensive models consistently outperform unstructured loops running frontier models: Singer et al. [7] show that their multi-host framework with Claude Haiku 3.5 surpasses a strong single-agent baseline built on Claude Sonnet 4, and separate work demonstrates that GPT-4o-mini exceeds GPT-4o at web exploitation once a deterministic finite-state machine governs agent behaviour [6]. This architectural primacy motivates close attention to structural design rather than model selection in the work described here.

The emergence of capable LLM agents has opened a genuinely new question for cybersecurity: *can an LLM agent carry out penetration testing tasks autonomously, without step-by-step human direction?*

### 1.1.4 The Emergence of Autonomous VAPT Research

A growing body of research — developed between 2023 and 2025 — has answered parts of this question affirmatively, and in doing so has produced the quantitative baseline against which this thesis positions itself.

Early work by Fang et al. [8] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities, establishing that the basic capability exists. A later study by the same group collected a benchmark of 15 real-world one-day vulnerabilities and found that GPT-4, when given a CVE

description, achieved a pass@5 success rate of 87% — while every other tested model (GPT-3.5, eight open-source models) and every open-source scanner achieved 0% [5]. Crucially, without the CVE description, GPT-4's pass@5 rate collapsed to 7%, establishing an important early distinction: exploiting a vulnerability whose location and nature the agent already knows differs fundamentally from discovering an unknown vulnerability independently. This discovery gap is the central challenge that subsequent systems attempt to close.

Zhu et al. [1] addressed this gap directly by introducing a hierarchical team of planning and task-specific agents (HPTSA), which was the first system reported to autonomously exploit real zero-day vulnerabilities — vulnerabilities for which no CVE description is provided. HPTSA achieved a pass@5 rate of 42% and a pass@1 rate of 18% on a 14-CVE benchmark, representing a 4.3× improvement over a no-description single-agent baseline on pass@1. These figures are the primary comparative baseline for the evaluation described in this thesis.

Deng et al. [9] identified two recurring failure modes in single-agent systems: first, context loss over long sessions, which causes the agent to lose awareness of previously attempted attack paths (Finding 3); and second, a depth-first, recency-biased search pattern, in which the agent anchors to the most recently discussed surface and fails to return to earlier discoveries (Finding 4). These two modes together explain why single-agent systems consistently underperform on targets that require coordinated, multi-step attack chains. Later systems address these failure modes through multi-agent architectures [1, 4], declarative task abstractions [7], and planning-augmented designs [10].

### 1.1.5 Motivation for This Thesis

The starting point for this work was a close reading of this body of literature to determine whether it had already addressed the core challenges of autonomous VAPT, and if not, where exactly the remaining gaps lie. The review spans single-agent

exploitation studies, hierarchical multi-agent frameworks, planning-augmented approaches, and the principal benchmark suites used to assess autonomous VAPT agents (surveyed in full in Chapter 2).

Two recurring failure modes surfaced across independently developed systems: agents that search broadly over a target's attack surface do so without an explicit model of which vulnerabilities depend on which others; agents that reason about such dependencies do so on pre-enumerated, curated weakness sets that are not assembled dynamically during a live mission. These two failure modes appear to be complementary rather than coincidental, and they are the gaps this thesis has chosen to investigate. The investigation is at an early stage — the following chapters describe the direction, the proposed design, and the evidence that motivates it, rather than results already obtained.

## 1.2 Problem Statement

Autonomous VAPT systems face two recurring failure modes that have been independently measured across multiple benchmark studies. The two modes are complementary rather than coincidental: exploration breadth and dependency-aware planning are not separable capabilities. A system that explores widely without a model of which vulnerabilities depend on which others will revisit dead-end paths and waste its iteration budget on routes that are unreachable without prior steps. Conversely, a system that reasons about dependencies only over a pre-supplied weakness set cannot operate on an unfamiliar attack surface where no such set exists in advance. Both quantitative failure modes are described below.

**Failure Mode 1 — Insufficient exploration.** On CVE-Bench [2], a benchmark of 40 critical-severity (CVSS $\geq$ 9.0) real-world web-application CVEs, the strongest reported agent achieves a pass@5 success rate of only 13% (one-day setting) and 10% (zero-day setting). CVE-Bench's failure-mode breakdown (Table 5 of Zhu et al. 2) identifies *insufficient exploration* as the dominant cause of failure across every agent and evaluation setting, with exploration-failure rates ranging from 37.5% to 80.0%,

as illustrated in Figure 1.1. The failure is not merely a matter of insufficient effort: CVE-Bench defines it specifically as an agent's failure to locate the exploitable endpoint, even when given a high-level vulnerability description. Agents converge early on a narrow set of candidate paths and do not return to probe the remainder of the attack surface.

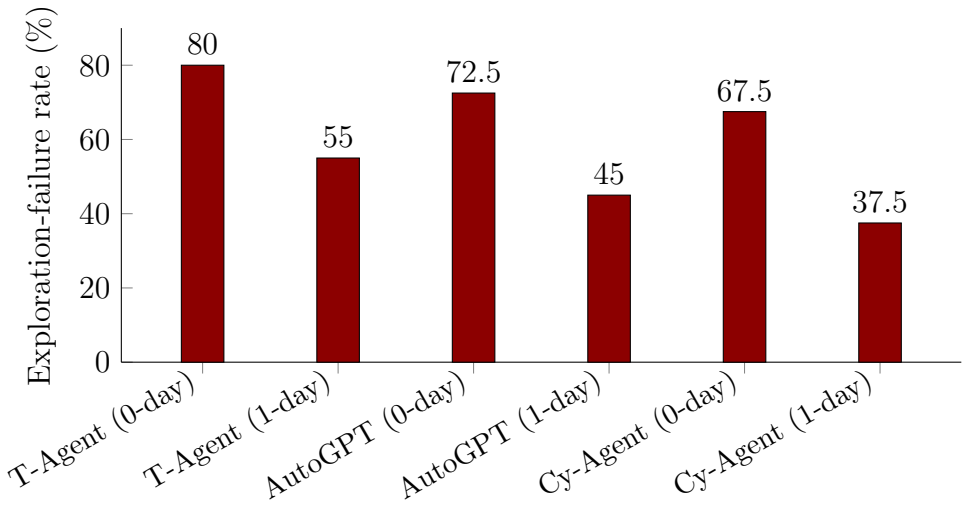


FIGURE 1.1: Exploration-failure rate as the dominant CVE-Bench failure mode, by agent and evaluation setting. T-Agent is the hierarchical multi-agent framework from Zhu et al. [1]; data from Table 5 of Zhu et al. [2].

**Failure Mode 2 — The dependency-reasoning gap.** PentestEval [3] decomposes the penetration testing pipeline into six sequential stages — Information Collection (IC), Weakness Gathering (WG), Weakness Filtering (WF), Attack Decision-Making (ADM), Exploit Generation (EG), and Exploit Revision (ER) — and evaluates nine state-of-the-art LLMs on each. The two weakest stages are WG (average Jaccard similarity 0.29 against expert ground truth) and ADM, the planning stage that requires reasoning about prerequisite relationships between candidate weaknesses. ADM's average Spearman rank correlation against expert judgement is 0.25, consistent across all nine evaluated models (range: 0.07 for GPT-3.5-Turbo to 0.34 for Qwen-Max): no tested model achieves meaningful alignment with expert attack-decision reasoning. The ground-truth-injection ablation in PentestEval quantifies how much end-to-end performance improves when each stage's output is replaced with expert annotations, applied cumulatively, as summarised in Table 1.1.

TABLE 1.1: PentestEval ground-truth-injection ablation: cumulative end-to-end success rate as expert ground truth replaces each stage's LLM output, from left to right [3].

| Configuration | End-to-end success | Marginal gain |
|---|---|---|
| SMP (baseline, no ground truth) | 0.31 | – |
| + GT Weakness Gathering (WG) | 0.50 | +0.19 |
| + GT Weakness Filtering (WF) | 0.53 | +0.03 |
| + GT Attack Decision-Making (ADM) | 0.67 | **+0.14** |

Two observations from Table 1.1 are critical for motivating this thesis. First, the current end-to-end ceiling without any ground truth is 0.31 — the SMP baseline that simply executes the five stages in sequence with LLM outputs throughout. Second, ADM delivers the largest single-stage marginal gain (+0.14) of any stage, and this gain is measured on top of an already ground-truthed WG and WF pipeline, not in isolation. The gap between 0.31 and 0.67 — a 36-point range — is the portion of end-to-end performance attributable, in this benchmark, to better dependency reasoning alone. Any dynamically constructed dependency structure that an agent builds from its own discovery output, without expert annotation, will approximate rather than match the ground-truth ceiling, so the realistic target sits somewhere within this range rather than at its upper bound.

**The compound gap.** The two failure modes together identify a gap that appears unaddressed in the reviewed literature. Systems built for broad exploration of an unfamiliar attack surface — HPTSA [1] and VulnBot [4] — dispatch task lists from a central planner without an explicit model of which weaknesses must precede others. The one system in the reviewed literature that models dependencies explicitly — CHECKMATE [10], using PDDL-style planning operators — requires the operator to enumerate relevant weaknesses before the planning phase begins; it is not designed to operate on an unknown attack surface where no weakness list is available in advance. A third gap is cross-surface evaluability: every system in the reviewed set undergoes evaluation against a single attack-surface family, making it unclear how well any architectural decision generalises beyond the surface on which it was measured.

The problem this thesis investigates is therefore: *can a single LLM-orchestrated multi-agent architecture combine open-ended exploration of an unfamiliar attack surface with a dynamically constructed, prerequisite-aware dependency model, and if so, does the combination produce a measurable improvement in end-to-end task success when evaluated against independently published benchmarks spanning more than one attack-surface family?* Whether this is achievable within the constraints of a six-month research programme, and whether the improvement is large enough to be meaningful, are empirical questions that the remaining chapters describe a plan for answering.

## 1.3 Objectives

Guided by the three gaps identified in Section 1.2, the objectives below define the research questions this thesis investigates. They are working hypotheses, not completed results; the exact scope and design of each will be revised as implementation and early experiments provide evidence that the literature review alone cannot. Each objective maps to a specific measurable claim that will be evaluated against the benchmarks described in Section 1.4.

1. **Dependency-aware exploration via the VDG.** To investigate whether representing candidate vulnerabilities as a **Vulnerability Dependency Graph (VDG)** — a directed graph in which edges encode prerequisite relationships between candidate weaknesses, and node selection is governed by an Upper Confidence Bound (UCB) policy that balances exploitation of high-confidence paths with exploration of unvisited ones — produces a measurable improvement in pass@5 exploitation success over flat-dispatch baselines on CVE-Bench [2] and PentestEval [3].

2. **Dual-layer knowledge representation.** To design and implement a **Dual-Layer World Model** that cleanly separates what an agent has empirically confirmed about a target (the Evidence Layer, recording tool outputs and verified state facts) from what it has inferred but not yet confirmed (the Analysis

Layer, hosting the VDG and ordinal evidence scores). Separating these layers is intended to allow independent diagnosis of where a mission fails — during discovery, during dependency inference, or during exploit generation — and to prevent unverified hypotheses from contaminating the agent's operational state.

3. **Four-layer agent orchestration.** To specify and implement a four-layer orchestration hierarchy — Mission Planner (goal decomposition and strategy), Team Manager (phase coordination and sub-agent dispatch), Specialist Agents (surface-specific execution roles), and an Execution and Validation Layer (tool invocation and result parsing) — drawing on the architectural patterns that recur across the strongest-performing systems in the reviewed literature [1, 4, 7], and to evaluate whether this structure improves performance relative to both single-agent and flat multi-agent baselines.

4. **Cross-mission memory and transfer.** To examine whether a **Cross-Mission Memory** module — which retains and selectively replays successful attack strategies from prior engagements, keyed by target technology fingerprint — produces a measurable benefit on targets that share underlying technology, without introducing the negative-transfer risk that arises when a strategy valid against one version of a component is applied to an incompatible version. This objective is treated as secondary: whether it becomes a core research question or supporting infrastructure depends on what early design work reveals.

5. **Multi-surface evaluation.** To evaluate the full system against at least two independently published, oracle-backed benchmarks covering distinct attack-surface families — specifically CVE-Bench (web application CVEs) [2], PentestEval (stage-level penetration testing) [3], PrediQL (GraphQL APIs) [11], and Incalmo MHBench (multi-host Active Directory environments) [7] — with per-surface reporting that distinguishes shared architectural gains from surface-specific results, rather than reporting a single aggregated number.

## 1.4 Scope of the Project

Based on the literature reviewed in Chapter 2, this project bounds itself by one governing rule: work proceeds only with attack surfaces for which a dedicated, reusable, oracle-backed benchmark already exists in the reviewed literature, rather than constructing new benchmarks independently. This constraint keeps the evaluation grounded and reproducible, and it ensures that any performance claim this thesis makes can be independently verified against a fixed, published test suite. The scope may narrow further once early implementation provides more information about what is achievable within the remaining timeline. Table 1.2 summarises the three attack-surface families currently considered in scope, the primary benchmarks that govern each, and the vulnerability types each family contributes to the evaluation.

TABLE 1.2: Attack surfaces in scope, their primary governing benchmarks, and representative vulnerability types.

| **Attack surface** | **Primary governing benchmark(s)** | **Representative vulnerability types** |
|---|---|---|
| Web application (HTTP/HTML) | CVE-Bench — 40 critical-severity CVEs, oracle-backed pass/fail [2]; PentestEval — 346 tasks across 12 real-world scenarios, stage-level scoring [3] | SQLi, XSS, CSRF, SSRF, SSTI, LFI, file-upload RCE, IDOR, authentication bypass, privilege escalation |
| GraphQL APIs | PrediQL — 6 real GraphQL APIs, dependency-aware oracle [11] | Schema abuse, dependency-chain injection, batched-auth bypass, IDOR |
| Multi-host / Active Directory | Incalmo MHBench — 40 networked environments, declarative task oracle [7] | Lateral movement, credential reuse, privilege escalation across host chains |

Results from the single-agent and hierarchical-team baselines in Fang et al. [5] and Zhu et al. [1] — 87% and 42% pass@5 respectively on their own curated suites — serve as comparative reference points rather than primary evaluation targets: RedGrid is not re-evaluated on those closed suites, but their reported numbers contextualise what the state of the art was before the benchmarks listed in Table 1.2 existed.

**What is currently left out.** General REST API exploitation is not assessed directly: no standardised, reusable REST API benchmark comparable to CVE-Bench or PrediQL appears in the reviewed literature. Techniques from the REST API testing literature — dependency inference between API calls, response-feedback pruning — may inform internal design choices, but REST-API-specific performance claims are not part of the evaluation plan. Binary exploitation, physical- and network-layer attacks, and social engineering are also outside the current scope for the same reason: no oracle-backed benchmark for these surfaces appears in the reviewed literature, and the governing rule of this project requires one. Should a suitable benchmark emerge during the thesis period, the scope section will be revised accordingly.

## 1.5 Expected Contributions

The following reflects the direction this thesis considers worth pursuing, framed as working hypotheses rather than results already in hand. None of the items below has yet been implemented or empirically assessed; each represents a claim to be tested against the benchmarks described in Section 1.4. The framing will be revised as implementation and early experiments provide evidence the literature review alone cannot.

- **A proposed system architecture.** The primary architectural contribution is a four-layer orchestration hierarchy — Mission Planner, Team Manager, Specialist Agents, and Execution/Validation Layer — combined with a Dual-Layer World Model that separates empirically confirmed target state (Evidence Layer) from inferred but unconfirmed hypotheses (Analysis Layer, hosting the VDG). This design synthesises structural patterns that recur independently across the strongest-performing systems in the reviewed literature [1, 4, 7] and addresses the mismatch identified in Sections 1.2 and 1.3: no reviewed system combines both layers simultaneously. Whether this architecture delivers a performance improvement over flat-dispatch baselines is an

empirical question; the architecture itself, together with its formal specification, is a contribution independent of that outcome.

- **Dependency-aware exploration via the VDG.** The primary empirical hypothesis is that a **Vulnerability Dependency Graph (VDG)** — a dynamically constructed directed graph in which edges encode prerequisite relationships between candidate weaknesses, and node selection is governed by a UCB policy — improves end-to-end exploitation success relative to flat-dispatch baselines. The quantitative opportunity is bounded: the current best end-to-end baseline without ground truth is 0.31 (PentestEval SMP); the upper bound with perfect attack-decision-making is 0.67 (PentestEval GT-ADM ablation). The target is an improvement somewhere within this range, measured on CVE-Bench [2] and PentestEval [3]. No position on the expected magnitude of improvement is available with confidence at this stage.

- **Cross-mission memory and selective strategy reuse.** A secondary hypothesis is that a **Cross-Mission Memory** module — retaining and selectively replaying successful attack strategies from prior engagements, keyed by target technology fingerprint — produces a measurable benefit on targets sharing underlying technology, without inducing the negative-transfer risk discussed in Section 1.6. Whether this becomes a core research contribution or supporting infrastructure depends on what early design work reveals; it is not a prerequisite for the primary architectural and VDG contributions.

- **Multi-surface evaluation with separated per-surface reporting.** Independently of whether the above hypotheses hold, this thesis intends to evaluate the proposed system against at least two independently published benchmarks spanning distinct attack-surface families — CVE-Bench and PentestEval (web applications), PrediQL (GraphQL APIs) [11], and Incalmo MHBench (multi-host Active Directory environments) [7] — with standardised oracles and per-surface reporting. Every reviewed system undergoes evaluation on a single attack-surface family, making it impossible to distinguish architectural gains from surface-specific tuning effects. Multi-surface evaluation with separated

reporting is therefore a methodological contribution in itself, regardless of the numeric outcome.

The engineering infrastructure that enables the above — structured inter-agent communication protocols, retry-and-diagnose validation loops, deterministic tool-call logging, and benchmark harness integration — is a necessary precondition for reproducible evaluation rather than an independent research claim. It is listed here for completeness and will be documented in detail in Chapter 4.

## 1.6 Challenges

The challenges listed below surfaced repeatedly during the literature review and represent specific risks to the validity of the proposed research. They are stated here explicitly, rather than deferred to implementation, so that each can be assigned a mitigation strategy before work begins rather than after a problem is encountered.

1. **Inferring dependencies without ground truth.** The VDG's edges are constructed from LLM inference over live discovery output, not from expert annotation. This is the single riskiest component of the design: PentestEval [3] shows that even with a perfect, expert-supplied weakness set, the best LLM achieves only 0.34 Spearman correlation on attack decision-making. LLM-inferred edges on a noisy, dynamically discovered set will be less accurate still. *Mitigation:* before relying on VDG inference in any live evaluation, a pilot study will measure Spearman alignment between VDG-inferred dependency rankings and PentestEval's expert-annotated ground truth on Scenarios 1–12, and the resulting accuracy figure will be reported alongside all end-to-end results.

2. **Sandbox results may not transfer to the real world.** Fang et al. [8] found only 1 exploitable XSS in 50 real-world candidate sites (2%), against

73.3% in a matched sandbox. All evaluation in this thesis is against published, oracle-backed benchmarks rather than live production systems. *Mitigation:* every reported result will explicitly state the benchmark, its construction method, and whether it uses real deployed software (CVE-Bench, PrediQL) or purpose-built lab environments (PentestEval, MHBench), so that readers can calibrate the real-world transferability of any number.

3. **Hallucination in exploit generation and tool use.** CVE-Bench identifies tool misuse as a failure mode in 47.5% of Cy-Agent zero-day attempts [2]; PentestEval documents four specific failure patterns in exploit generation — missing parameters, loss of critical payload syntax, hallucinated dependencies, and incorrect tool flags [3]. These failures occur at the Execution/Validation Layer and will directly affect any performance number this thesis reports. *Mitigation:* the Execution/Validation Layer will include a retry-and-diagnose loop that captures tool-call errors, classifies them by failure type, and feeds structured error context back to the generating Specialist before counting a failure — consistent with PentestEval's Exploit Revision stage design.

4. **Context loss and recency bias across long missions.** Deng et al. [9] document two compounding failure modes in long-session agents: context loss (the agent loses awareness of previously attempted paths as the conversation grows) and depth-first recency bias (the agent anchors to the most recently discussed surface and fails to return to earlier discoveries). Any multi-phase VAPT mission of realistic length will trigger both. *Mitigation:* the Dual-Layer World Model separates confirmed facts (Evidence Layer, persistent) from inferred hypotheses (Analysis Layer/VDG, updated incrementally) so that context compaction in the LLM's active window does not corrupt the agent's factual record of what has been attempted.

5. **Benchmark scale varies significantly across surfaces.** PrediQL covers only 6 GraphQL APIs, against CVE-Bench's 40 CVEs and MHBench's 40 environments. PrediQL's APIs are real production deployments (not synthetic),

which gives them high ecological validity, but the small N limits statistical confidence. *Mitigation:* conclusions drawn from PrediQL results will be labelled exploratory; confidence intervals will be reported where sample size permits; and PrediQL findings will be presented as hypothesis-generating rather than hypothesis-confirming.

6. **Reused strategies could introduce negative transfer.** A Cross-Mission Memory entry for a successful exploit against one version of a technology could be actively harmful if replayed against an incompatible version — for example, a payload crafted for WordPress 5.9 breaking against WordPress 6.4. The reviewed papers do not address this risk. *Mitigation:* Cross-Mission Memory entries will store the target technology fingerprint (application name, version range, framework) alongside the strategy; replay will be gated by a version-compatibility check before the strategy is offered to the Team Manager.

7. **Sensitivity to configuration choices.** The VDG's UCB exploration constant, the edge-confidence pruning threshold, and the Evidence Layer's staleness window are all tunable parameters. Reporting results from a single, un-swept configuration risks overfitting to a fortunate setting. *Mitigation:* a sensitivity analysis will be conducted on at minimum the UCB exploration constant and the VDG edge-confidence threshold before the final evaluation configuration is fixed; the swept range and chosen value will be reported explicitly.

8. **Honesty about what generalises across surfaces.** Testing across web, GraphQL, and multi-host surfaces with the same system does not mean the entire system is unmodified across surfaces. Specialist Agent pools, tool adapters (ZAP/sqlmap for web; schema-aware fuzzer for GraphQL; BloodHound/credential tools for Active Directory), and oracle interfaces will necessarily differ. *Mitigation:* per-surface results will be reported separately, and a component table will explicitly identify which modules are shared across all surfaces and which are surface-specific, so that claims about generalisation are bounded by what is actually shared.

## 1.7 Organization

This report reflects the first stage of work on this thesis, and its organization serves as a plan for the full document the authors expect to submit in approximately six months, not a description of chapters that already exist in complete form. Chapter 2 presents a review of a focused selection of papers on autonomous VAPT, organized by architectural approach, and identifies the gaps in prior work that motivated the direction described in Sections 1.2 and 1.3; this chapter is the most developed at present. Chapter 3 presents the proposed system design as it develops — currently sketched at a high level and subject to revision as implementation proceeds. Chapter 4 lays out the work breakdown, timeline, and milestones for the remaining months, and stays current as work progresses. Chapter 5 specifies and, eventually, reports on the evaluation plan once an implementation exists to assess. Chapter 6 summarises the completed work, and that chapter takes substantive form only much later in the thesis timeline.

# Chapter 2

# Literature Review and Related Work

## 2.1 Domain Overview

This chapter reviews the recent literature on LLM-based autonomous penetration testing and identifies, at a preliminary level, the patterns and gaps that motivate the research direction described in Chapter 1. The review covers a focused selection of eleven papers, chosen as the most directly relevant to the research question from a broader set of papers read during the preparation of this report. Table 2.1 provides an overview of these eleven papers; the sections that follow discuss them in more depth.

Autonomous penetration testing — the use of LLM agents to independently discover and exploit weaknesses in a target system without step-by-step human direction — has emerged as an active research area only in the last two to three years. The domain sits at the intersection of two older fields: Vulnerability Assessment and Penetration Testing (VAPT), which traditionally relies on human operators following structured methodologies (reconnaissance, weakness identification, exploitation, and reporting), and LLM agent research, which studies how language models can plan, use tools, and act toward a goal over sequential steps. The work described

in this report, RedGrid, sits in this intersection: its focus is how an LLM-driven multi-agent system can carry out the VAPT workflow autonomously against realistic, benchmarked targets.

A useful way to frame the domain is around two capabilities that an autonomous VAPT agent must exhibit simultaneously. The first is *exploration*: the agent must search a large space of candidate weaknesses across an unfamiliar attack surface without prior knowledge of which one is exploitable. The second is *prerequisite reasoning*: many real exploits are not single actions but chains, where one weakness (for example, an information disclosure or a misconfigured endpoint) must precede a second, more consequential weakness before it becomes reachable. As the literature reviewed in this chapter shows, most existing systems prioritize one of these capabilities at the expense of the other — a pattern that motivates the closer reading in the sections that follow.

The sections that follow first survey the systems and benchmarks from this selection (Section 2.2), then draw out the recurring architectural patterns across them (Section 2.3), and finally identify, at a preliminary level, where the reviewed literature appears to leave a gap (Section 2.4).

## 2.2 Existing Systems

This section discusses the eleven papers selected in Table 2.1, grouped by the category of contribution they represent.

### 2.2.1 Early Single-Agent Approaches

The earliest work in this space used a single LLM agent operating in a Reason + Act (ReAct) loop against a target. Fang et al. [8] showed that a single GPT-4 agent with browser access could autonomously exploit simplified web vulnerabilities, establishing that the basic capability exists. A later study by the same group [5] collected a benchmark of 15 real-world one-day vulnerabilities and found that GPT-4, when

TABLE 2.1: The eleven papers selected for review, with their main focus and relevance to this research.

| No. | Paper | Main Focus | Why Relevant |
|---|---|---|---|
| 1 | Fang et al. (2024) — LLM Agents can Autonomously Hack Websites [8] | Single-agent web exploitation | Foundational: first demonstration of autonomous web attacks; establishes sandbox vs. real-world gap |
| 2 | Fang et al. (2024) — LLM Agents can Autonomously Exploit One-day Vulnerabilities [5] | Single-agent CVE exploitation | Shows GPT-4 can exploit real CVEs; key result: exploiting $\neq$ discovering |
| 3 | Zhu et al. (2024) — Teams of LLM Agents can Exploit Zero-Day Vulnerabilities [1] | Hierarchical multi-agent VAPT | First zero-day autonomous exploitation; introduces hierarchical planner + specialist design |
| 4 | Deng et al. (2024) — PentestGPT [9] | Single-agent with reasoning/parsing split | Identifies context-loss failure mode in long sessions; motivates structured memory and handoff |
| 5 | Kong et al. (2025) — VulnBot [4] | Multi-agent, role-specialised | Representative of team-dispatch architecture; shows flat dispatch without dependency modeling |
| 6 | Wang et al. (2025) — CHECKMATE [10] | LLM agents + classical planning | Explicit prerequisite modeling via PDDL; evaluated on curated scenarios only |
| 7 | Singer et al. (2025) — Incalmo [7] | Multi-host LLM red teaming | Extends autonomous VAPT to multi-host / Active Directory environments; declarative task API |
| 8 | Liu et al. (2025) — PrediQL [11] | GraphQL API testing with LLMs | Shows schema-based dependency reasoning for a non-web attack surface |
| 9 | Zhu et al. (2025) — CVE-Bench [2] | Benchmark for web CVE exploitation | Provides empirical evidence that insufficient exploration is the dominant failure mode |
| 10 | Yang et al. (2025) — PentestEval [3] | Stage-level benchmark | Shows attack decision-making (prerequisite reasoning) is the weakest stage in current systems |
| 11 | Wang et al. (2025) — Survey on LLM-based Autonomous Agents [6] | General LLM agent survey | Provides the conceptual and architectural vocabulary underlying all domain-specific systems |

given the CVE description, exploited 87% of them — while every other tested model and every open-source scanner achieved 0%. Crucially, without the CVE description, GPT-4's success dropped to 7%, establishing an important early distinction: exploiting a vulnerability whose location the agent already knows differs fundamentally from discovering an unknown vulnerability independently. This distinction motivates much of the later work in the field.

Deng et al. [9] extended single-agent design with an explicit reasoning/generation/-parsing split aimed at reducing context loss over long testing sessions — a failure mode documented in detail there, and appearing as a concern across many later systems. PentestGPT ran on the HackTheBox and VulnHub platforms, which provide machine-level penetration testing challenges. The work identifies a fundamental tension: agents need a long conversational history to maintain awareness of what they have already tried, but long histories inflate the context window and degrade reasoning quality. This tension between memory breadth and reasoning focus is one of the recurring architectural challenges in the literature.

### 2.2.2 Multi-Agent and Team-Based Orchestration

A second line of work replaces the single agent with a team of cooperating agents, motivated by the observation that a single flat loop struggles to scale to wide or unfamiliar attack surfaces. Zhu et al. [1] introduced a hierarchical planner (HPTSA) that dispatches task-specific sub-agents and was the first system reported to exploit real zero-day vulnerabilities autonomously — vulnerabilities for which no CVE description is provided. This result is significant: it shows that, with the right architecture, LLM agents can discover vulnerabilities rather than merely exploit known ones.

Kong et al. [4] propose VulnBot, a comparable multi-agent framework with explicit role specialisation — separating reconnaissance, planning, and exploitation into distinct agent roles. Like HPTSA, VulnBot dispatches tasks from a central planner to specialist agents, but without explicitly modeling prerequisite relationships between

candidate vulnerabilities. The tasks go out as a flat list, and agents work through them in sequence without a formal model of what depends on what.

A general point that emerges from these multi-agent systems — and that the broader survey of the field supports [6] — is that architecture, not model scale, appears to be the dominant variable in autonomous agent performance. A well-structured pipeline tends to outperform an unstructured loop even when the latter uses a more capable base model. This architectural primacy motivates close attention to the structural design of RedGrid.

### 2.2.3 Planning-Augmented Approaches

A smaller set of papers pairs LLM agents with a more classical planning component, aiming to give the agent an explicit model of prerequisites between actions rather than relying purely on the LLM's implicit reasoning. Wang et al. [10] combine LLM agents with classical planning techniques (PDDL-style operators) for penetration testing, and report stronger performance on the specific scenarios where this planning applies. These systems typically require enumerating the relevant weaknesses in advance, which limits their applicability to open-ended discovery on an unfamiliar attack surface — a limitation discussed further in Section 2.4.

### 2.2.4 Non-Web Attack Surfaces and Benchmarks

The final group of papers covers two non-web attack surfaces and the two benchmark studies that provide the quantitative evidence for the failure modes described in Chapter 1.

Singer et al. [7] extend autonomous penetration testing to multi-host and Active Directory-style network environments, where the agent must perform lateral movement, credential reuse, and privilege escalation across a chain of networked hosts. This is a qualitatively different challenge from single-target web testing: the agent must maintain awareness of its progress across an evolving network of compromised

and uncompromised hosts. Incalmo addresses this partly through a structured, declarative task vocabulary that keeps the agent's planning language manageable. Liu et al. [11] take a different direction, targeting GraphQL APIs specifically. Their work is notable for introducing schema-based dependency reasoning within the API testing context: a GraphQL schema encodes relationships between types and fields that the agent can exploit to guide its testing strategy, drawing on the same underlying intuition about prerequisite relationships that motivates this work.

On the evaluation side, Zhu et al. [2] introduce CVE-Bench, a benchmark of 40 critical (CVSS $\geq$ 9.0) real-world web-application CVEs with oracle-backed pass/fail signals. Its failure-mode analysis provides the clearest available quantitative evidence that insufficient exploration — not reasoning quality — forms the primary bottleneck in current agents. Yang et al. [3] introduce PentestEval, which decomposes the penetration testing pipeline into discrete stages and measures agent performance at each stage. Its finding that attack decision-making — reasoning about which weakness to pursue next, given what the agent has discovered so far — ranks as the weakest stage provides complementary evidence to CVE-Bench: where CVE-Bench identifies exploration breadth as the failure, PentestEval identifies prerequisite reasoning as the failure, and together they suggest that the two capabilities need joint attention.

## 2.3 Comparative Analysis

Table 2.2 summarises the eleven reviewed systems and benchmarks along four dimensions that recurred throughout the literature review: whether the system uses a single agent or a coordinated team, whether it models dependencies between candidate weaknesses explicitly, whether it retains any form of memory across sessions, and the benchmarks used in its evaluation.

A few patterns are visible from this comparison. First, systems that scale to a wide or unfamiliar attack surface (HPTSA, VulnBot) dispatch tasks flatly, without an explicit model of which weaknesses depend on which others. Second, the one system

TABLE 2.2: Comparison of systems and benchmarks reviewed, across four recurring architectural dimensions.

| System / Work | Architecture | Dependency Modeling | Cross-Session Memory | Benchmarks |
|---|---|---|---|---|
| Fang et al. [8] | Single agent (ReAct) | None | None | Simplified websites |
| Fang et al. [5] | Single agent (ReAct) | None (given CVE desc.) | None | 15 one-day CVEs |
| HPTSA [1] | Hierarchical planner + task agents | None (flat dispatch) | None | 15 zero-day CVEs |
| PentestGPT [9] | Single agent, split reasoning/parsing | Implicit (LLM only) | None | HTB/VulnHub machines |
| VulnBot [4] | Multi-agent, role-specialised | None (flat dispatch) | None | HTB-style scenarios |
| CHECKMATE [10] | Agent + classical planner | Explicit, pre-enumerated | None | Curated scenarios |
| Incalmo [7] | Multi-host orchestration | Partial (host/-credential graph) | None | MHBench (40 envs.) |
| PrediQL [11] | LLM-guided fuzzer | Schema-derived | None | 6 GraphQL APIs |
| CVE-Bench [2] | Benchmark (no agent) | N/A | N/A | 40 critical web CVEs |
| PentestEval [3] | Benchmark (modular pipeline) | Explicit (ground-truth injected) | None | 12 real-world scenarios |
| Wang et al. [6] | Survey | N/A | N/A | General (survey) |

that does model dependencies explicitly (CHECKMATE) undergoes evaluation on curated scenarios where relevant weaknesses are pre-enumerated, and it's not clear from the reviewed paper how well this approach extends to open-ended discovery. Third, cross-session memory — the ability for an agent to keep and reuse what it learned in a previous engagement against a similar target — is absent from every system in the table. Finally, the two benchmark papers (CVE-Bench and PentestEval) each provide stage-level failure analysis that points in the same direction: existing systems are weakest precisely at the combination of broad exploration and structured prerequisite reasoning that handling a wide, unfamiliar attack surface

demands.

These observations are preliminary, and the following section discusses their implications for the research direction this thesis pursues.

## 2.4 Research Gap

The literature reviewed in this chapter suggests, tentatively, that autonomous VAPT systems have so far addressed one of two capabilities at a time, rather than both simultaneously. Table 2.3 summarises this observation.

TABLE 2.3: Observed split in the reviewed literature between exploration-oriented and dependency-reasoning-oriented systems (preliminary reading).

| **Area** | **What the reviewed literature shows** | **Observed limitation / open area** |
|---|---|---|
| Exploration breadth | HPTSA and VulnBot scale to wide, unfamiliar attack surfaces using hierarchical team dispatch | Neither models prerequisite relationships between discovered weaknesses; both dispatch tasks as flat lists |
| Prerequisite / dependency reasoning | CHECKMATE introduces classical planning with explicit prerequisites; PentestEval shows that ground-truth attack decision-making (ADM) produces the largest single-stage improvement (+0.14) | Both require weaknesses to be pre-enumerated; applicability to open-ended discovery is unclear from the reviewed papers |
| Cross-session memory | No system in the reviewed set retains and reuses strategies across back-to-back missions | An open area that may warrant investigation, though its value relative to the above two concerns remains unclear |
| Cross-surface evaluation | Every reviewed system undergoes evaluation against a single attack-surface family | No system evaluates the same architecture across web, GraphQL, and multi-host surfaces simultaneously |

On one side, the reviewed literature shows that systems built for broad exploration of an unfamiliar attack surface — HPTSA, VulnBot, and similar team-dispatch architectures — do not appear to model prerequisite relationships between candidate weaknesses explicitly. They search widely, but without a formal notion of which weaknesses must precede others. On the other side, systems that do reason

explicitly about such dependencies, including CHECKMATE's classical-planning layer and PentestEval's ground-truth attack decision-making configuration, require scenarios that list the relevant weaknesses in advance. How well these approaches scale to open-ended discovery on a wide, unfamiliar surface remains unclear from the reviewed papers.

This split appears in how the two most detailed benchmark studies in this selection report their results. CVE-Bench's own failure analysis attributes the majority of unsuccessful attempts on its hardest real-world CVEs to insufficient exploration rather than to reasoning quality. PentestEval's stage-level breakdown finds that the attack-decision-making stage — reasoning about which weakness to pursue next given what the agent has already discovered — consistently ranks as the weakest stage across the systems it evaluates. Taken together, these two findings point, at a preliminary stage of the review, at two sides of what may be the same underlying gap: the reviewed literature does not yet appear to contain a system that combines broad, open-ended exploration with an explicit, dynamically constructed model of dependencies between discovered weaknesses.

To be clear, this reflects an early stage reading of the literature. The gap described above serves as a working hypothesis to guide the rest of the thesis, rather than as a finalized problem statement. The architecture and approach of RedGrid will develop and receive justification more carefully as the work progresses.

## 2.5 Summary

This chapter has reviewed eleven papers directly relevant to autonomous, LLM-driven penetration testing, covering early single-agent systems, multi-agent orchestration frameworks, planning-augmented approaches, non-web attack surfaces, and the benchmark studies that provide the quantitative evidence for the field's current limitations. A comparative reading of this literature points, tentatively, to a recurring split between systems optimised for broad exploration and systems optimised

for dependency-aware reasoning, with the reviewed literature not appearing to contain a system that addresses both simultaneously, and with few systems assessed across more than one attack-surface family.

As this report reflects an early stage of the thesis, the survey above serves as a snapshot of work completed so far rather than a closed literature review. Over the coming months, the review deepens — in particular around the planning-augmented systems and the question of cross-session memory — and the observations in Section 2.4 motivate a more precisely scoped research question and approach as the thesis develops.

# Chapter 3

# Proposed Methodologies

## 3.1 System Overview

## 3.2 System Requirements

## 3.3 Proposed Architecture

## 3.4 System Workflow

## 3.5 Tools and Technologies

# Chapter 4

# Execution Plan

## 4.1 Work Breakdown Structure

## 4.2 Project Timeline

## 4.3 Milestones

## 4.4 Resource Requirements

## 4.5 Risk Assessment

# Chapter 5

# Experimental Results

## 5.1 Evaluation Plan

# Chapter 6

# Conclusions

## 6.1 Summary

## 6.2 Expected Deliverables

## 6.3 Future Works

# Bibliography

[1] Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. Teams of LLM agents can exploit zero-day vulnerabilities. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026. arXiv:2406.01637.

[2] Yuxuan Zhu, Antony Kellermann, Dylan Bowman, Philip Li, Akul Gupta, Adarsh Danda, Richard Fang, Conner Jensen, Eric Ihli, Jason Benn, Jet Geronimo, Avi Dhir, Sudhit Rao, Kaicheng Yu, Twm Stone, and Daniel Kang. CVE-Bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025.

[3] Ruozhao Yang, Mingfei Cheng, Gelei Deng, Tianwei Zhang, Junjie Wang, and Xiaofei Xie. PentestEval: Benchmarking LLM-based penetration testing with modular and stage-level design. Preprint, 2025.

[4] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. VulnBot: Autonomous penetration testing for a multi-agent collaborative framework. arXiv preprint arXiv:2501.13411, 2025.

[5] Richard Fang, Rohan Bindu, Akul Gupta, and Daniel Kang. LLM agents can autonomously exploit one-day vulnerabilities. arXiv preprint arXiv:2404.08144, 2024. v2, cs.CR.

[6] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhi-Yuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei

Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2025. arXiv:2308.11432.

[7] Brian Singer, Keane Lucas, Lakshmi Adiga, Meghna Jain, Lujo Bauer, and Vyas Sekar. Incalmo: An autonomous LLM-assisted system for red teaming multi-host networks. arXiv preprint arXiv:2501.16466, 2025.

[8] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. LLM agents can autonomously hack websites. arXiv preprint arXiv:2402.06664, 2024.

[9] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, Philadelphia, PA, 2024. USENIX Association.

[10] Lingzhi Wang, Xinyi Shi, Ziyu Li, Yi Jiang, Shiyu Tan, Yuhao Jiang, Junjie Cheng, Wenyuan Chen, Xiangmin Shen, Zhenyuan Li, and Yan Chen. Automated penetration testing with LLM agents and classical planning. arXiv preprint arXiv:2512.11143, 2025.

[11] Shaolun Liu, Sina Marefat, Omar Tsai, Yu Chen, Zecheng Deng, Jia Wang, and Mohammad A. Tayebi. PrediQL: Automated testing of GraphQL APIs with LLMs. arXiv preprint arXiv:2510.10407, 2025.